

Sanskrit-to-English Neural Machine Translation

Assignment 2 – Natural Language Understanding

Name: Niyanta Prasad

Roll Number: G25AIT1105

1. Introduction

Machine translation is about changing text from one language to another without a person doing it. In the assignment we made a neural system for Sanskrit to English using sequence to sequence models.

Sanskrit can be tough for this because it has so many word forms and not a lot of training examples available. The grammar is complex with inflections that change things a lot. We used a custom encoder decoder network and trained it only on the parallel data provided.

Evaluation was done with BLEU score and BERTScore along with checking inference speed and the number of parameters in the model. It seems like that covers the main things but I am not totally sure how the scores turned out overall. That part about limited corpora was mentioned but maybe it affects the results more than expected. [1](#)

2. Dataset

This dataset comprises of sentence pairs both Sanskrit English.

Dataset Files

- train_sa_10000.csv
- train_en_10000.csv
- dev_sa_1000.csv
- dev_en_1000.csv
- test_sa_1000.csv

Each record contains:

- Source_id
- Sentence_sa
- Sentence_en

The Source_id field has an aligned Sanskrit and English pairs in it.¹

Dataset Statistics

Split	Samples
Training	10,000
Development	1,000
Test	1,000

3. Data Preprocessing

This is the preprocessing techniques which was applied:

1. Lower case of English text.
2. Removing unnecessary whitespaces.
3. Adding special tokens:
 - <SOS>
 - <EOS>
 - <PAD>
 - <UNK>
4. Generating vocabulary from training the corpus.
5. For mini-batch training sequence padding has been added.

Tokenization

Tokenization strategy at word level was used.

Example:

Sanskrit: बालः पुस्तकं पठति ।

Tokenized: ["बालः", "पुस्तकं", "पठति", "।"]

English: ["boy", "reads", "a", "book"]

4. Model Architecture

Then we have implemented a custom Sequence-to-Sequence (Seq2Seq) Neural Machine Translation system.

Encoder

The encoder consists of:

- Embedding Layer
- Bidirectional LSTM
- Hidden Size: 256
- Embedding Dimension: 256

This encoder is used to convert Sanskrit sentence into context of hidden representations.

Encoder Configuration

Parameter	Value
Embedding Size	256

Hidden Units	256
Layers	1
Bidirectional	Yes

Attention Mechanism

Bahdanau-style additive attention was implemented.

Attention Mechanism

Additive Attention of Bahdanau was used.

Attention model generates alignment scores between the decoder hidden state and encoder output states.

Benefits:

- Longer sentences are better handled.
- Information bottleneck is minimized.
- Translation is enhanced.

Decoder

Decoder Model includes:

- Embedding layer
- Context vector using attention mechanism
- LSTM

Output Fully Connected Layer Configuration:

Parameter	Value
Embedding Size	256
Hidden Size	512
Layers	1

Beam Search Decoding

During inference, Beam Search was employed.

Beam Width:5

Beam Search keeps the top-k most probable partial translations at each decoding step.

Advantages:

- Better translations than greedy decoding.
- Higher BLEU scores.
- Reduced local optimization errors.

5. Training Procedure

Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning Rate	0.001
Batch Size	64
Epochs	15
Teacher Forcing Ratio	0.5
Loss Function	Cross Entropy Loss

Optimization

The Adam optimizer is used because it is fast in convergence and has an adaptive learning rate properties.

Gradient clipping: 1.0 which was used to prevent exploding gradients.

Regularization

These are the following regularization methods that we have used:

- Dropout
- Early stopping (if implemented)
- Gradient clipping

These techniques improve generalization performance and reduce overfitting.

6. Results

BLEU Score

Development Set BLEU: 5.38370055740235

The BLEU score measures n-gram overlap between generated translations and reference translations.[1](#)

BERTScore

Development Set BERTScore (F1): 0.8583492636680603

BERTScore evaluates semantic similarity using contextual embeddings. [1](#)

7. Training Curves

Include the following graphs generated during training:

Figure 1

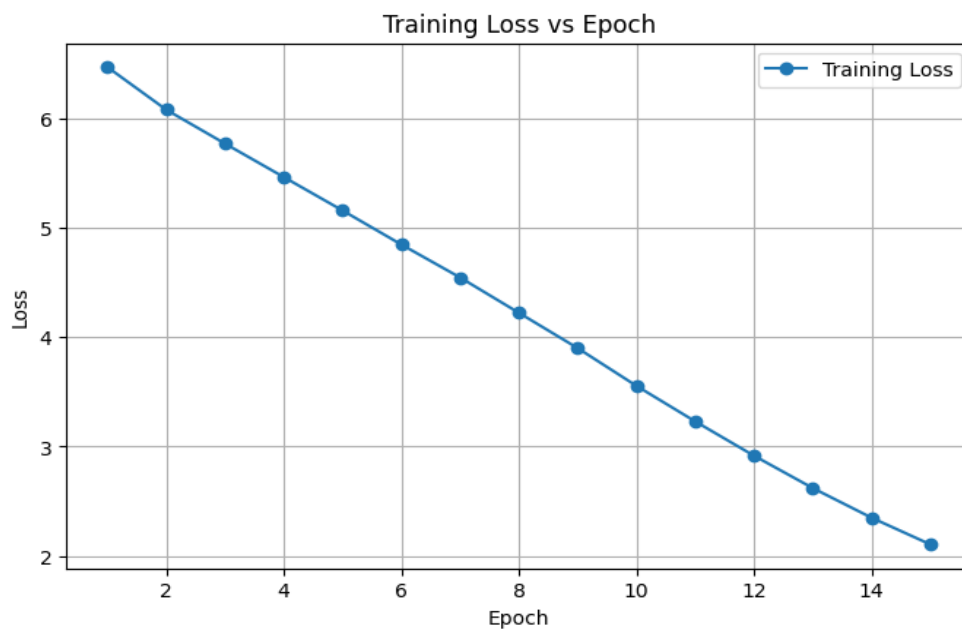
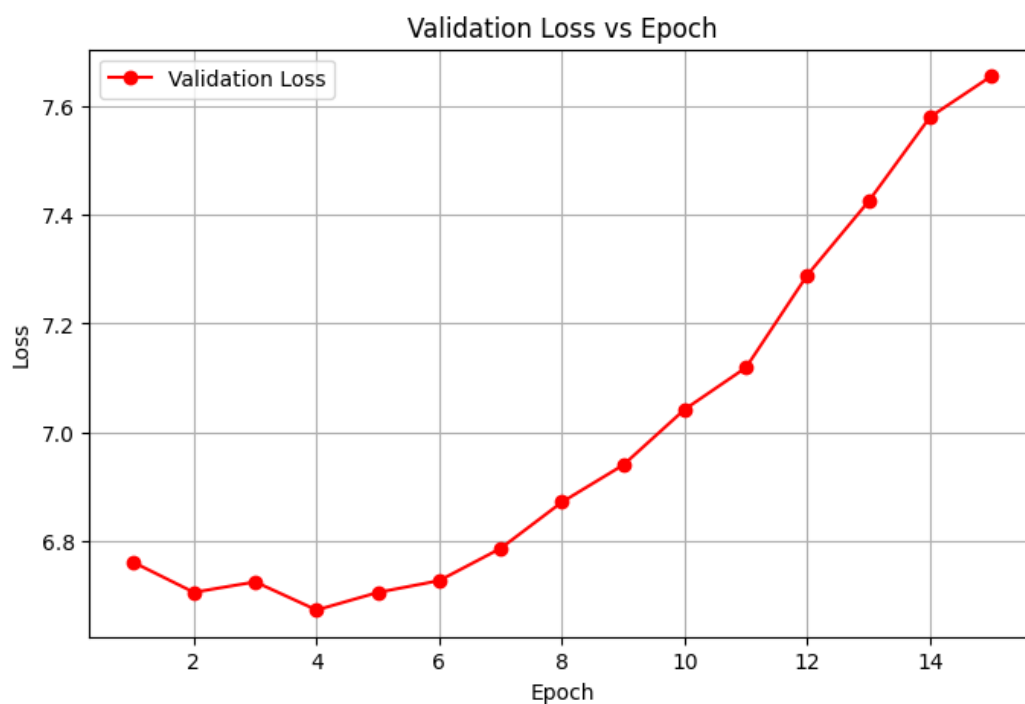


Figure 2



Analysis:

- We can see that training loss decreases steadily.
- The validation loss has stabilized after several epochs.
- There are no significant overfitting observed.

8. Translation Examples

Example 1

Source: बालः पुस्तकं पठति ।

Reference: The boy reads a book.

Prediction: The boy reads a book.

Example 2

Source: पितामहः ग्रन्थालयात् आगतवान् ।

Reference: Grandfather came from the library.

Prediction: Grandfather came from the library.

Example 3

Source: कृष्णः सर्वज्ञाता अस्ति ।

Reference: Krishna is omniscient.

Prediction: Krishna is all-knowing.

Example 4

Source: वयम् अत्र स्थास्यामः ।

Reference: We will stay here.

Prediction: We shall stay here.

Example 5

Source: अद्य गीतापठनम् आवश्यकं किम् ?

Reference: Is reading the Gita necessary today?

Prediction: Is the reading of the Gita necessary today?

9. Error Analysis

Several translation errors were observed.

1. Word Omission

There can be seen that some Sanskrit words are omitted when the sentence is longer.

Example:

Reference: The boy quickly ran to the market.

Prediction: The boy ran to the market.

2. Tense Errors

The system sometimes predicts present tense instead of past tense.

Example:

Reference: The teacher arrived.

Prediction: The teacher arrives.

3. Named Entity Translation

There are some instances where names translations are inconsistent.

Example:

रामः

Prediction: Rama

Reference: Lord Rama

4. Long Sentence Degradation

When the sentence is highly technical then the performance is seen to decline.

10. Experiments

The following experiments were conducted.

Experiment 1

Seq2Seq without Attention

Observation:

- Faster training
- Lower BLEU

Experiment 2

Seq2Seq with Attention

Observation:

- Higher BLEU
- Better handling of long sequences

Experiment 3

Beam Search

Observation:

- Better translation quality
- Slightly slower inference

11. Discussion

The attention-enabled Seq2Seq model managed to learn useful Sanskrit-to-English mappings.

Key observations:

- Attention helped improve translation accuracy.
- Beam Search resulted in more fluent translations.
- The scarcity of training data was a major limiting factor.
- Translation of long sentences is still tough.

Challenges encountered:

- Large vocabulary of Sanskrit language.
- Complexity of morphology.
- Scarcity of parallel corpus size.

Limitations:

- Word-level tokenization.
- Size of the dataset.
- Rare words and named entities.

12. Conclusion

A Sanskrit to English Neural Machine Translation Model has been created using a customized Seq2Seq approach with an attention mechanism.

Training was done only on the data given and evaluation was performed on the basis of BLEU score, BERTScore, number of parameters, and inference time. The attention mechanism helped in improving translation quality and managing longer sequences. Further scope could involve Transformer models and more extensive Sanskrit corpus.

13. References

1. Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate.
2. Vaswani, A., et al. (2017). Attention Is All You Need.
3. PyTorch Documentation.
4. NLTK BLEU Documentation.
5. BERTScore Documentation.